Randomised Algorithms

Sahil Mukesh Deo

May 29, 2026

Contents

Chapter 1

Introduction

1.1 What even is a Randomised Algorithm?

A randomised algorithm is an algorithm that makes random choices during its execution. Unlike deterministic algorithms, whose behaviour is completely determined by the input, the behaviour of a randomised algorithm may vary between different executions on the same input.

Typically, the algorithm uses a random number generator (rng for short) to decide between multiple possible actions. As a result, quantities such as running time[1], memory usage, or even correctness[2] can become random variables (though usually not all of them at the same time).

The performance of a randomised algorithm is therefore analysed using probability and expectation. If T_{run} denotes the runtime of the algorithm, we are usually interested in quantities such as $T_{algo} = E[T_{run}]$ over all possible random choices made by the algorithm. [3]

Randomised algorithms are often used for the following purposes:

- avoid adversarial worst case inputs (common in competitive programming)
- simplify algorithm design and implementation
- obtain polynomial-time approximate solutions to NP problems (the field of heuristic algorithms)

1.2 Implementation of Randomness in C++

Lets make things more concrete with some actual code and implementation details! Modern C++ provides pseudo-random[4] number generators through the <random> library. A commonly used generator is mt19937, which is based on the Mersenne Twister algorithm.

```
#include <bits/stdc++.h>
mt19937 rng(chrono::steady_clock::now().time_since_epoch().count()); //using time as a seed
```

[1] Such algorithms are called Las Vegas algorithms.

[2] Such an algorithm is called a Monte Carlo algorithm. The fact that correctness itself may become probabilistic can initially seem undesirable, especially in critical systems where absolute correctness is expected. However, in many practical situations, a sufficiently small error probability is considered acceptable. For example, if an algorithm can be shown to fail with probability at most 10^{-30}, then the chance of failure may already be smaller than the probability of random hardware faults such as memory bit flips caused by cosmic radiation, which programmers typically ignore during ordinary software development.

[3] T_{algo} can still be a random variable over the input - so this may **not** be the same as average case performance of the algorithm T_{avg}, which is an expected value over all possible inputs. But for cleanliness of presentation, we will often ignore this distinction and use these terms interchangeably, more often taken to mean T_{avg} than T_{algo}.

[4] Since computers are deterministic, they rely on parameters like atmospheric noise and temperature for randomness. The details are irrelevant for us... We will take pseudo random as true random for our purposes

This creates an object rng with seed[5] as current time which will be passed as a parameter to other functions. It has the capability to produce "truly" random 32-bit integers.

Generating Random Integers

To generate a random integer uniformly from a range $[L, R]$, we use

```
uniform_int_distribution<int>uid(L,R);

int x=uid(rng);
int y=uid(rng);
.
.
.
```

Each generation takes around constant time.

Generating Random Real Numbers

To generate a real number uniformly from an interval $[x, y)$, we use

```
uniform_real_distribution<double>urd(x,y);

double x=urd(rng);
double y=urd(rng);
.
.
.
```

This also takes constant time per generation.

Generating a Random Permutation

A random permutation of an array (in-place modification) can be generated using `shuffle`:

```
shuffle(a.begin(),a.end(),rng);
```

Internally, this does something like:

```
for(int i=n-1;i>0;i--)
{
    int j=random integer from [0,i];
    swap(a[i],a[j]);
}
```

(Try to think why all permutations are equally likely with this method!)

[5]Seed values are used in pseudo random generators as a way to recreate the generated random numbers. This can be used for debugging a randomised algorithm. If you don't care about that, set it to the current time in milliseconds or something similar and that should be good enough!

Implementing Random Decisions

Sometimes, an algorithm must choose between multiple actions with specified probabilities. For example, let three actions X,Y,Z be such that they are executed with probabilities:

$$\Pr(X) = \frac{1}{5}, \qquad \Pr(Y) = \frac{3}{5}, \qquad \Pr(Z) = \frac{1}{5}$$

One simple way to implement this is to generate a random integer uniformly from 1 to 5:

```
uniform_int_distribution<int>uid(1,5);

int v=uid(rng);

if(v==1)
{
    //do X
}
else if(v<=4)
{
    //do Y
}
else
{
    //do Z
}
```

thus implementing the required probabilities.

Chapter 2

Algorithmic Performance

For algorithms whose performance[1] depends on the input size or other input parameters, we usually estimate the number of elementary operations performed as a function of the input size n or the maximum value of input A_{max}, usually denoted by A. This gives a mathematical way to compare algorithms independent of hardware or implementation details.

A classical example is Bubble Sort. The algorithm repeatedly traverses the array, comparing adjacent elements and swapping them whenever they are in the wrong order. After each pass, the largest remaining element moves to its correct position near the end of the array.

With some thought it can be proved that the total number of swaps performed by Bubble Sort is exactly equal to the number of inversions N_{inv} in the input array. (An inversion is a pair of indices (i, j) such that $i < j$ but $a_i > a_j$.) By taking swaps to be one operation taking constant time, we can estimate runtime of Bubble sort to be $T_{run} \propto N_{inv}$

2.1 Worst Case Analysis

In worst case analysis, we determine the maximum possible number of operations over all valid inputs of size n. For Bubble Sort, the worst case occurs when the array is sorted in reverse order, where every pair is inverted.

$$N_{inv,max} = \binom{n}{2} = \frac{n(n-1)}{2}$$

Worst case analysis is useful because it guarantees an upper bound on the running time regardless of the input provided.

2.2 Average Case Analysis

In average case analysis, we compute the expected number of operations assuming that all valid inputs are equally likely.

For Bubble Sort, this corresponds to analysing the expected number of inversions in a random permutation of size n. For every pair of indices (i, j) with $i < j$, the probability that the pair forms an inversion is

$$\Pr\big((i,j) \text{ is an inversion}\big) = \frac{1}{2}$$

[1] Although this section focuses primarily on running time, the same methods of analysis can also be applied to other resources such as memory usage.

Define the indicator[2] variable I_{ij} for each pair (i, j) as follows:

$$I_{ij} = \begin{cases} 1, & \text{if } (i,j) \text{ is an inversion} \\ 0, & \text{otherwise} \end{cases}$$

$$N_{\text{inv}} = \sum_{1 \le i < j \le n} I_{ij}$$

Using linearity of expectation[3],

$$\mathbb{E}[N_{\text{inv}}] = \mathbb{E}\left[\sum_{1 \le i < j \le n} I_{ij}\right] = \sum_{1 \le i < j \le n} \mathbb{E}[I_{ij}]$$

$$\mathbb{E}[I_{ij}] = 1 \cdot \frac{1}{2} + 0 \cdot \frac{1}{2} = \frac{1}{2}$$

$$\implies \mathbb{E}[N_{\text{inv}}] = \sum_{1 \le i < j \le n} \frac{1}{2} = \frac{1}{2}\binom{n}{2} = \frac{n(n-1)}{4}$$

So notice that if we randomise the input sequence, T_{algo} becomes equal to T_{avg} thus improving performance by a factor[4] of 2.

2.3 Time Complexity Analysis

We are sometimes interested in comparing runtimes like $T_{run} = n^2$ to $T_{run} = 100nlogn$. To compare growth rates, we use asymptotic notation:

$$T(n) = \Theta(f(n))$$

means that $T(n)$ grows asymptotically at the same rate as $f(n)$.

My favourite way to think about time complexity is as follows: we try to find the "simplest" function $f(n)$ such that

$$\lim_{n \to \infty} \frac{T(n)}{f(n)} = k, \qquad k \in \mathbb{R}_{>0}$$

That is, $T(n)$ and $f(n)$ grow at the same asymptotic rate up to a positive constant factor.

For example, Bubble Sort performs on the order of n^2 operations in both the average and worst case, so its time complexity is

$$T(n) = \Theta(n^2)$$

Asymptotic analysis allows us to compare algorithms by their rate of growth rather than implementation-dependent constants.

This allows us to say that A: $T_{run} = n^2$ is worse than B: $T_{run} = 100nlogn$ since A has $T(n) = \Theta(n^2)$ and B has $T(n) = \Theta(nlogn)$ even though for some small n the first one may well be better, but for small n the performance doesn't really matter anyway...

[2] Refer Appendix A.4

[3] Refer Appendix A.1

[4] Randomising the input itself takes about n operations, so more rigorously we would say an asymptotic factor of 2. In most cases we will be able to neglect the time required to randomise the input.

Chapter 3

The Classics

3.1 Quick Sort

Quick Sort is one of the most important examples of a randomised algorithm in practice. The basic idea is very simple:

1. Choose a pivot element.
2. Partition the array into:
   - elements smaller than the pivot,
   - elements equal to the pivot,
   - elements greater than the pivot.

   This takes n comparisions with our pivot
3. Recursively sort the smaller and larger parts.

A deterministic implementation may always choose the first or last element as pivot. Unfortunately, this can lead to very poor performance on adversarial inputs. For example, if the array is already sorted and we always choose the first element as pivot, the recursion becomes extremely unbalanced:

$$(n-1) + (n-2) + \cdots + 1 = \Theta(n^2)$$

Thus deterministic Quick Sort has worst case complexity

$$T(n) = \Theta(n^2)$$

The standard randomised version instead chooses the pivot uniformly at random.

```
uniform_int_distribution<int>uid(0,a.size()-1);
int pivot=uid(rng);
swap(a[pivot],a[r]);
```

This tiny modification dramatically changes the behaviour of the algorithm. Intuitively, it becomes very unlikely that we repeatedly choose extremely bad pivots.

Runtime Analysis

Let $T(n)$ denote the expected runtime of randomised Quick Sort on an array of size n over all possible inputs.

Suppose the pivot finally ends up at position k, where $0 \leq k \leq n-1$. Then the recursive subproblems have sizes k and $n-1-k$. Since partitioning itself takes n operations, we obtain the recurrence

$$T(n) = T(k) + T(n-1-k) + n$$

Because the pivot is chosen uniformly at random, every value of k is equally likely. Taking expectation over all random choices made by the algorithm gives

$$T(n) = n + \frac{1}{n}\sum_{k=0}^{n-1}\Big(T(k) + T(n-1-k)\Big)$$

By symmetry,

$$\sum_{k=0}^{n-1} T(k) = \sum_{k=0}^{n-1} T(n-1-k)$$

therefore

$$T(n) = \Theta(n) + \frac{2}{n}\sum_{k=0}^{n-1} T(k)$$

$$S(n) = \sum_{k=0}^{n} T(k)$$

Then

$$T(n) = S(n) - S(n-1)$$

and the recurrence becomes

$$S(n) - S(n-1) = cn + \frac{2}{n}S(n-1)$$

$$S(n) = \left(1 + \frac{2}{n}\right) S(n-1) + cn$$

$$\frac{S(n)}{(n+1)(n+2)} = \frac{S(n-1)}{n(n+1)} + \frac{cn}{(n+1)(n+2)}$$

Summing this recurrence telescopically gives

$$\frac{S(n)}{(n+1)(n+2)} = \Theta\left(\sum_{k=1}^{n}\frac{1}{k}\right) = \Theta(\log n)$$

Therefore,

$$S(n) = \Theta(n^2 \log n)$$

Finally,

$$T(n) = S(n) - S(n-1) = \Theta(n \log n)$$

Hence the expected runtime of randomised Quick Sort is

$$T(n) = \Theta(n \log n)$$

This analysis demonstrated a powerful technique to find the expected runtime - recursion on input size. It was a bit messy and lengthy, but sometimes yields results very quickly.

Another possible way is through indicator variables. Observe that any pair of elements is compared at most once during the entire execution of Quick Sort.

Let

$$I_{ij} = \begin{cases} 1, & \text{if elements } i, j \text{ are compared} \\ 0, & \text{otherwise} \end{cases}$$

Then the total number of comparisons is

$$C = \sum_{1 \le i < j \le n} I_{ij}$$

Two elements are compared only if one of them becomes the first pivot chosen among all elements lying between them in sorted order. It can be shown that

$$\Pr(I_{ij} = 1) = \frac{2}{j - i + 1}$$

Therefore,

$$\mathbb{E}[C] = \sum_{1 \le i < j \le n} \frac{2}{j - i + 1} = \Theta(n \log n)$$

Hence randomised Quick Sort has expected complexity

$$T(n) = \Theta(n \log n)$$

3.2 Quick Select

The Problem: find the k-th smallest element of an unsorted input array

An obvious method is to sort the array using Merge Sort in $\Theta(n \log n)$ and just access the k-th element - this is overall $\Theta(n \log n)$ and is deterministic. Then how could randomness help?

Quick Select is an algorithm used to find the k-th smallest element of an array.

Similar to Quick Sort, the algorithm chooses a pivot and partitions the array into three categories by doing n-1 comparisons between the pivot and every other element as before:

- elements smaller than the pivot,
- elements equal to the pivot,
- elements greater than the pivot.

However, unlike Quick Sort, we recurse into only **one** side:

- if the pivot ends up at position k, we are done,
- if k lies to the left, recurse left,
- otherwise recurse right.

Thus each step discards a significant fraction of the array. This is a similar approach to Binary Search.

Expected Runtime

Suppose partitioning takes n time where array size is n. Let T(n) be the runtime. If the pivot is the i-th smallest element, then

$$T(n) = n+$$

which is a geometric series:

$$T(n) = \Theta(n)$$

Thus randomised Quick Select runs in expected linear time.

Chapter 4

Online Decision Algorithms

What is an Online Algorithm?

An online algorithm is an algorithm that processes its input piece-by-piece in a serial fashion, without having the entire input available from the start. In some problems, this doesn't matter – the obvious solution is online since it requires a single pass over the array only. But in some problems, the online requirement makes the problem much harder and forces us to make decisions without knowing the future.

Online decision algorithms have randomness involved in their analysis for non adversarial inputs. Further, the goal of the algorithm usually is not deterministic optimality, but rather to maximise the probability of success or to achieve a good competitive ratio[1].

4.1 The Dating Problem

Problem Statement: Consider the problem of finding the best partner. Assume that the number of people you will eventually date is fixed, i.e. some known value n. The people appear in a uniformly random order. After dating a person, you must immediately decide whether to commit to them forever or reject them permanently and continue searching. Figure out a strategy that maximises the probability of ending up with the best partner.[2]

We assume that while dating person i, we only know how they compare relative to the people we have already dated. So we could sort the partners till now in a total order if needed.

Solution: Most course materials, internet reels or shorts I have encountered this problem in directly jump to the famous strategy of:

> "Reject the first r people, then accept the first person afterwards who is better than everyone seen so far."

However, they often omit the proof of why an optimal strategy must necessarily have this threshold form. We first prove this fact.

Optimality Proof

Suppose we are currently considering the k-th person.

[1] Competitive ratio is a measure of how well an online algorithm performs compared to an optimal offline algorithm that has access to the entire input from the start. The term n-competitive is used when an algorithm achieves atleast $\frac{1}{n}$ times the "performance" of an optimal offline algorithm.

[2] This is a different problem from maximising the expected "value" of your partner. Here the objective is specifically to maximise the probability of selecting the single best partner among all n people.

If the current person is *not* the best among the first k people seen so far, then accepting them can never succeed: This is because there already exists an earlier person who is strictly better, and that earlier person has already been rejected. Hence an optimal strategy should only ever consider accepting people who are the best seen so far.

Now suppose the current person *is* the best among the first k people seen so far. Let $\boldsymbol{q_k}$ be the optimal probability of eventually selecting the globally best person if we reject the k-th person and continue optimally afterwards.

If we instead accept the current person immediately, what is the probability of success?

Since the ordering of the n people is uniformly random, each of the first k positions is equally likely to contain the globally best person. Conditioned on the fact that the current person is the best among the first k, the probability that they are actually the globally best person equals:

$$\frac{k}{n}$$

Therefore:

- accepting gives success probability $\frac{k}{n}$,
- rejecting gives success probability q_k.

Hence the optimal decision is to accept iff $\frac{k}{n} \geq q_k$[3].

Now observe that $\frac{k}{n}$ is increasing with k.

On the other hand, q_k is decreasing with k. To see this, notice that from time k, one valid strategy is:

> "Reject one more person no matter what, and then behave optimally from time $k+1$ onwards."

This strategy achieves success probability exactly q_{k+1}. Since q_k is the *optimal* achievable success probability starting from time k, we must have:

$$q_k \geq q_{k+1}$$

Since $\frac{k}{n}$ is increasing and q_k is decreasing, the optimal decision changes at most once from rejecting to accepting as k increases. Therefore there exists some threshold $0 \leq r \leq n$ such that:

- for $k \leq r$, rejecting is optimal,
- for $k > r$, accepting the first "best-so-far" person is optimal.

Hence every optimal strategy must be a threshold strategy. Note r=0 corresponds to accepting the first person, and r=n corresponds to rejecting everyone and ending up with no partner. Both of these are valid threshold strategies, though they are not very good ones[4].

[3] This is a common theme in online algorithms - the optimal decision at each step is determined by comparing the expected "value" of actions and choosing the one with maximum value. Here our "value" is the success probability. In game theory this is called "utility" instead.

[4] Though it may be argued that having no partner is the best choice - this is beyond the scope of this text and is left as an exercise for the reader.

Finding the Optimal Threshold

We now analyse the threshold strategy:

> Reject the first r people, then accept the first person afterwards who is better than everyone seen so far.

Suppose the globally best person appears at position k.
For our strategy to succeed:

1. we must have $k > r$,
2. among the first $k-1$ people, the best person must appear within the first r positions.

The first condition ensures that we do not automatically reject the best person.
The second condition ensures that no earlier candidate after position r triggers acceptance before the globally best person appears.
Now:

$$\Pr[\text{best person appears at position } k] = \frac{1}{n}$$

Further, among the first $k-1$ people, each position is equally likely to contain their maximum. Therefore:

$$\Pr[\text{maximum among first } k-1 \text{ lies in first } r] = \frac{r}{k-1}$$

Hence:

$$\Pr[\text{success}] = \sum_{k=r+1}^{n} \frac{1}{n} \cdot \frac{r}{k-1}$$

$$= \frac{r}{n} \sum_{k=r}^{n-1} \frac{1}{k}$$

Using the approximation:

$$\sum_{k=r}^{n-1} \frac{1}{k} \approx \ln\left(\frac{n}{r}\right)$$

we obtain:

$$P(r) \approx \frac{r}{n} \ln\left(\frac{n}{r}\right)$$

Let:

$$x = \frac{r}{n}$$

Then:

$$P(x) = x \ln\left(\frac{1}{x}\right)$$

Differentiating:

$$P'(x) = \ln\left(\frac{1}{x}\right) - 1$$

Setting:

$$P'(x) = 0$$

gives:

$$\ln\left(\frac{1}{x}\right) = 1$$

$$x = \frac{1}{e}$$

Thus the optimal strategy is:

> Reject approximately the first $\frac{n}{e}$ people, then accept the first person afterwards who is better than everyone seen so far.

The corresponding success probability is:

$$\boxed{\frac{1}{e}}$$

which is approximately 36.8%. Not bad at all!

Strictly speaking, we should also verify that this critical point indeed corresponds to a maximum rather than a minimum. One way is via the second derivative test:

$$P''(x) = -\frac{1}{x} < 0$$

for all $x > 0$, so $P(x)$ is concave and hence the critical point is necessarily a global maximum.

Alternatively, we can avoid calculus entirely by inspecting the behaviour near the boundaries. Observe:

$$P(x) = x \ln\left(\frac{1}{x}\right)$$

As $x \to 0$,

$$P(x) \to 0$$

since x decays faster than $\ln(1/x)$ grows.

Similarly, as $x \to 1$,

$$P(x) \to 0$$

because:

$$\ln(1) = 0$$

Since $P(x)$ is positive for $0 < x < 1$, rises away from 0, and eventually falls back to 0, the unique critical point at:

$$x = \frac{1}{e}$$

must correspond to the global maximum.

Behaviour for Small n

We used some limiting n approximations in finding the optimal threshold (I hope you noticed!). In case you were not planning on dating a tending-to-infinite number of people, you might be interested in the optimal strategy for small values of n. For finite n, the exact success probability for threshold r is:

$$\max_{0 \le r \le n} \left(\frac{r}{n} \sum_{k=r}^{n-1} \frac{1}{k} \right)$$

Computationally we can now easily find max for each n:

n	2	3	4	5	6	7
n/e	0.74	1.10	1.47	1.84	2.21	2.58
round(n/e)	1	1	1	2	2	3
Actual optimal r	**1**	**1**	**1**	**2**	**2**	**3**

The corresponding optimal success probabilities are:

n	2	3	4	5	6	7
Pr[success]	**0.500**	**0.500**	**0.458**	**0.433**	**0.428**	**0.414**

We observe that the optimal threshold follows the value suggested by $\frac{n}{e}$ even for relatively small values of n, and it will get closer and closer as n increases. The success probability is already around 41.4% for $n = 7$, and it will approach the limiting value of $\frac{1}{e} \approx 36.8\%$ as n grows.

4.2 The Parallel Processing Problem

Consider a setting with m identical machines processing incoming tasks. The tasks arrive one after another, and the arrival times are sufficiently close that we may assume they arrive within negligible time of each other.

Each incoming task must immediately be assigned to one of the m machines. Once a decision is made, it cannot be reversed. Thus, the algorithm has no knowledge of future tasks while making its decision.

The tasks assigned to a machine are processed sequentially and hence form a queue at that machine.

Suppose the incoming tasks have processing times

$$p_1, p_2, \ldots, p_n$$

where n itself is not known in advance.

The objective is to minimize the *makespan*, i.e. the finishing time of the busiest machine:

$$C_{\max} = \max_{1 \le i \le m} T_i$$

where T_i denotes the total processing time assigned to machine i. This is a natural objective since the overall processing time is determined by the machine that finishes last[5].

4.2.1 Greedy Algorithm

A natural strategy is the following greedy algorithm.

Whenever a new task arrives, assign it to the machine currently having the minimum total processing time.

> For every incoming job j with processing time p_j:
>
> 1. Find the machine with minimum current load.
> 2. Assign job j to that machine.

4.2.2 Performance Analysis

Interestingly, we do not actually have a polynomial time algorithm to find the optimal offline solution to this problem.

We will therefore use lower bounds on OPT (the optimal offline makespan) to derive an upper bound on the competitive ratio of the greedy algorithm.

Let ALG denote the makespan produced by the greedy algorithm.

Consider the task that finishes last in the greedy schedule. This may or may not be the last *assigned* task. Let its processing time be p.

Immediately before assigning this job, suppose the minimum machine load was L.

By our hypothesis that this machine will be the one that finishes last, the greedy makespan is

$$ALG = L + p$$

[5]This concept is similar to the Rate Determining Step (RDS) in the field of Chemical Kinetics

Hence the total load already assigned before this final job was at least:

$$mL$$

Including the final job, the total processing load is therefore at least:

$$mL + p$$

By Pigeonhole Principle, at least one machine must have load at least the average load, so –

$$OPT \geq \frac{mL + p}{m} = L + \frac{p}{m}$$

Since any valid schedule must process the job of size p,

$$OPT \geq p \implies OPT(1 - \frac{1}{m}) \geq p\left(1 - \frac{1}{m}\right)$$

Adding the two inequalities gives:

$$OPT(2 - \frac{1}{m}) \geq L + p = ALG$$

Hence,

$$\frac{ALG}{OPT} \leq 2 - \frac{1}{m}$$

Therefore the greedy online load balancing algorithm is $\left(2 - \frac{1}{m}\right)$-competitive, usually written as 2-competitive since the difference is negligible for large m.

4.2.3 Tightness of the Bound

The previous analysis showed that the greedy algorithm is $\left(2 - \frac{1}{m}\right)$-competitive. We now construct an example showing that the analysis cannot, in general, be improved beyond a factor of $\left(2 - \frac{1}{m}\right)$.

Consider $m = 4$ machines and the following sequence of incoming jobs:

$$1, 1, 1, \ldots, 1 \text{ (12 times)}, 4$$

The greedy algorithm will assign the first 12 jobs as follows:

Machine i	1	2	3	4
Jobs assigned	1, 1, 1	1, 1, 1	1, 1, 1	1, 1, 1

After processing these 12 jobs, each machine has load 3. The last job of size 4 will be assigned to any machine (say machine 1), resulting in a final makespan of 7. On the other hand, an optimal offline algorithm could have assigned the first 12 jobs as follows:

Machine i	1	2	3	4
Jobs assigned	1, 1, 1, 1	1, 1, 1, 1	1, 1, 1, 1	4

After processing these 13 jobs, each machine has load 4. Thus the optimal makespan is 4, and the competitive ratio is $\frac{7}{4}$, which is indeed our bound of $2 - \frac{1}{4}$.

Chapter 5

Data Structures

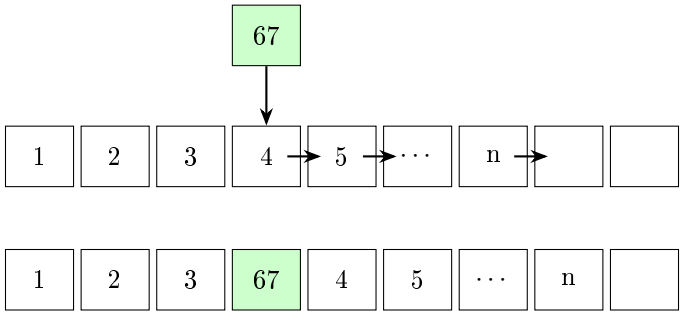
5.1 Skip Lists

Why not just use Arrays?

Suppose we store the following array in contiguous memory:

Suppose we wish to insert the element 67 between 3 and 4.

Since arrays occupy contiguous memory, every element beginning from 4 must be shifted one position to the right in order to create space for the new element.



Insertion into an array requires shifting $\Theta(n)$ elements in the worst as well as average case, so a single insertion is $\Theta(n)$ time complexity.

How are Linked Lists any better?

let us briefly recall the linked list data structure.

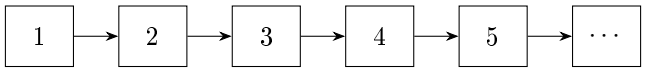
Unlike arrays, linked lists do not store elements contiguously in memory. Instead, each element stores:

- the value of the element, and
- a pointer to the next element.

A linked list containing the elements

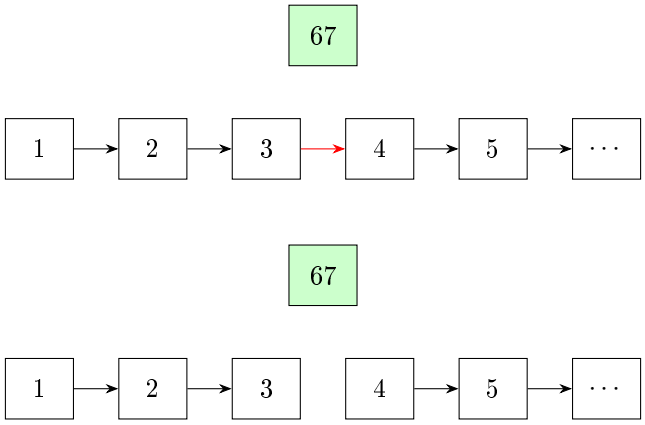
$$1, 2, 3, 4, 5, \cdots, n$$

may be represented as:

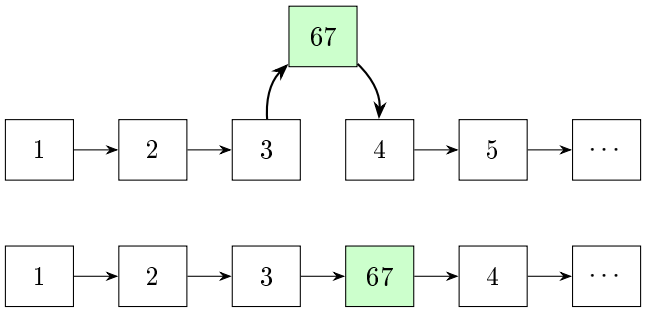


To insert an element 67 between say 3 and 4, we simply create a new node containing 67 and link it between 3 and 4.

Step 1: Remove the old connection



Step 2: Create new connections



Done!

We used a constant number of pointer operations (3) to insert an element into the list. Thus insertion is $\Theta(1)$ time complexity.

Why Skip Lists?

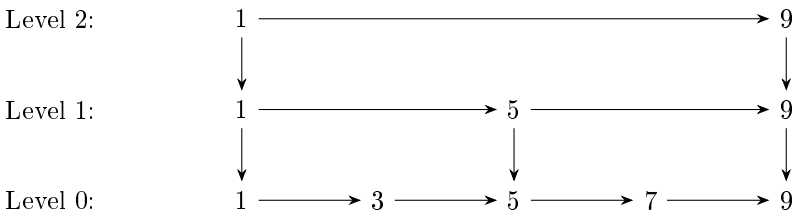
Although insertion is $\Theta(1)$ time complexity, searching for an element in linked lists is $\Theta(n)$ time complexity because we have to traverse the whole list. Even if the linked list is "sorted", we can not binary search since that requires random access in constant time, which a linked list can not provide.

So the idea of skip lists is to bypass that limitation by adding “express lanes” to a linked list.

Instead of traversing every element one-by-one, we maintain additional higher-level lists that skip[1] over many elements at once.

Consider the following skip list:

[1] Hence the name!



Higher levels contain fewer elements and allow us to jump across large portions of the list quickly.

Searching in a Skip List

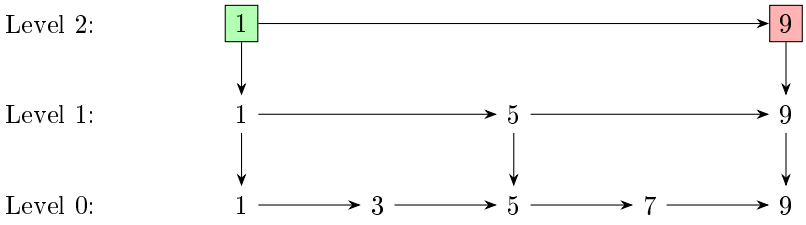
Suppose we wish to search for the element 7.

We begin at the top left level and move right while the next element is ≤ 7. Once that element is strictly greater than 7, we move down to the next level. Repeat until we reach the bottom level.

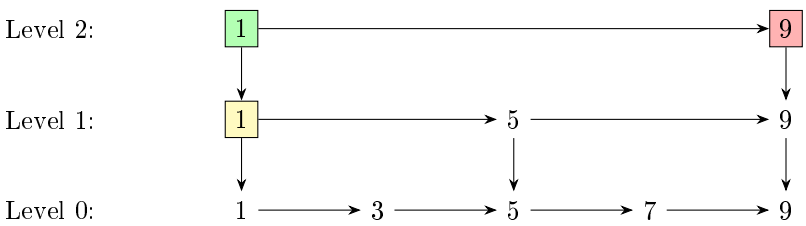
Pseudocode:

```
SEARCH(x):

current = top-left node
while current exists:

    while current.right exists
          and current.right.value <= x:
        current = current.right

    if current.value == x:
        return FOUND

    current = current.down

return NOT FOUND
```

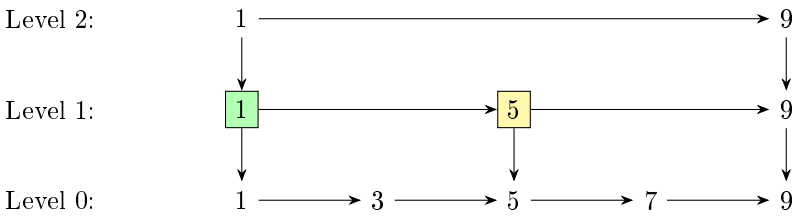
Step 1: Start at top, check right



Since on the right we have an element strictly greater than 7, we intend to move down.

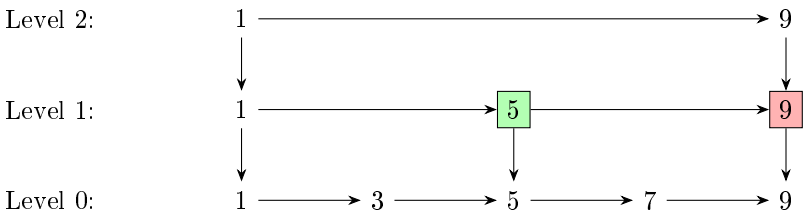


Step 2: Moved down, now check right

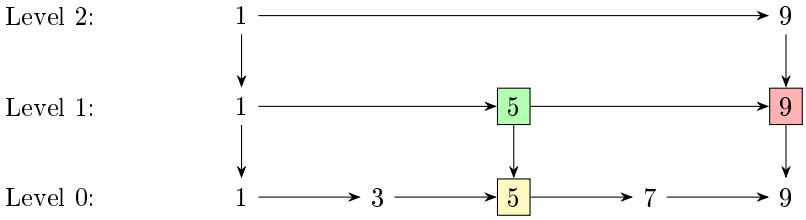


Since $5 \leq 7$, we intend to move right.

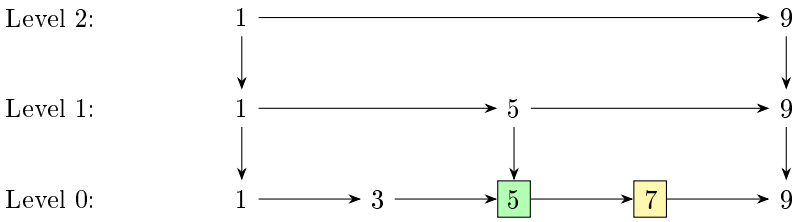
Step 3: Moved right, check right



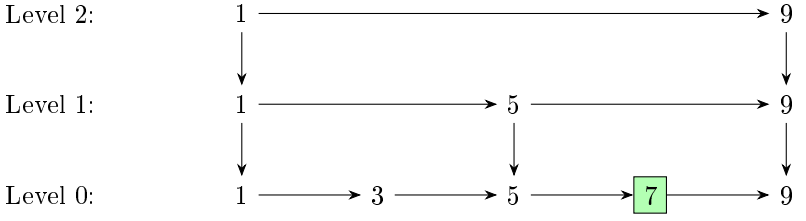
Since $9 > 7$, we intend to move down.



Step 4: Moved down, check right



Step 5: Moved right, found the target!



We move right to 7 and locate the desired element.

Why is the Expected Complexity $\Theta(\log n)$?

The logarithmic complexity of skip lists arises from the probabilistic structure of the levels.

Recall that:

- every node appears in level 0,
- each node independently gets promoted to the next level with probability $\frac{1}{2}$.

Consequently:

$$\Pr(\text{node reaches level } k) = \frac{1}{2^k}$$

Thus, the expected number of nodes in level k is:

$$\frac{n}{2^k}$$

For example:

Level	Expected Number of Nodes
0	n
1	$\frac{n}{2}$
2	$\frac{n}{4}$
3	$\frac{n}{8}$
$\vdots$	$\vdots$

Hence, the number of nodes decreases geometrically as we move upward.

Height of the Skip List The highest non-empty level is approximately the value h satisfying

$$\frac{n}{2^h} \approx 1$$

Solving:

$$2^h \approx n$$

Taking logarithms:

$$h \approx \log_2 n$$

Thus, the expected height of the skip list is:

$$\mathcal{O}(\log n)$$

Cost of Horizontal Movement Now consider the search procedure.

At each level:

- we move right several times,
- then move down one level.

A key probabilistic fact is that the expected number of right moves per level is bounded by a constant.

Intuitively:

- only about half the nodes survive to the next level,
- so long runs of horizontal movement become increasingly unlikely,

- and we quickly descend toward the target.

More formally, because each node is promoted independently with probability $\frac{1}{2}$, the expected gap between consecutive nodes at level k is approximately 2^k.

Thus, each higher level allows us to skip increasingly large portions of the list.

Total Search Complexity The search path therefore consists of:

- roughly $\mathcal{O}(\log n)$ downward moves,
- and only $\mathcal{O}(1)$ expected horizontal moves per level.

Hence:

$$\text{Expected Search Cost} = \mathcal{O}(\log n) \times \mathcal{O}(1) = \mathcal{O}(\log n)$$

Why is it randomised? One may wonder why skip lists use randomization at all. Could we not deterministically place:

- every second element in level 1,
- every fourth element in level 2,
- every eighth element in level 3,
- and so on?

While this would indeed support logarithmic search complexity, insertions and deletions would become expensive.

For example, inserting a new element near the beginning of the list changes which elements are the:

- second,
- fourth,
- eighth,
- etc.

elements of the list.

Consequently, many levels may require large-scale rebuilding.

Randomization avoids this problem by assigning heights independently to each node. Thus, updates are $\Theta(\mathit{height}) = \Theta(\log n)$ while the structure stays balanced in expectation.

Thus, randomized skip lists achieve logarithmic expected search complexity without requiring complicated balancing operations such as rotations or restructuring.

Chapter 6

Hashing

6.1 What even is Hashing?

A recurring theme in randomized algorithms is the idea of replacing a large object by a much smaller *fingerprint* or *hash* which approximately preserves equality.

Instead of comparing two complicated objects directly, we compare their hashes. If the hashes differ, the objects are certainly different. If the hashes are equal, the objects are *probably* equal.

The power of hashing comes from the fact that computing and comparing hashes is often significantly faster than comparing the original objects themselves. It may also be seen as a form of compression that preserves some information about the original object. In data structures like the `unordered set`, hashing helps to achieve constant time complexity (on average) within finite memory constraints.

Usually hashes are many-one functions, meaning that multiple different objects can have the same hash. Two inputs having the same hash is called a *collision*. The probability of collision depends on the specific hash function used and the distribution of inputs. A good hash function should minimize the probability of collisions for typical inputs.

6.2 Polynomial Verification

Suppose we are given two expressions:

$$P(x) = \prod_{i=1}^{k}(a_i x + b_i), Q(x) = \sum_{j=0}^{k} c_j x^j$$

We would like to verify whether

$$P(x) \equiv Q(x)$$

Naively expanding the product takes $\Theta(k^2)$ time, and then comparing the resulting polynomials takes $\Theta(k)$ time, so the total verification time is $\Theta(k^2)$.

A more efficient approach is to compute the hashes of the polynomials. Choose three random values

$$r_1, r_2, r_3$$

from the domain of the polynomial.

Define the hash of a polynomial to be the tuple

$$(P(r_1), P(r_2), P(r_3))$$

Similarly compute

$$(Q(r_1), Q(r_2), Q(r_3))$$

If the hashes differ at any coordinate, then the polynomials are certainly different.
If all three evaluations agree, then with high probability the polynomials are identical.

The reason this works is that the nonzero polynomial $P(x) - Q(x)$of degree at most d can have at most d roots in the domain from which we are choosing the random values.

If we are choosing random 32-bit integers, then the probability of a false positive (i.e. the polynomials are different but the hashes match) is at most $\left(\frac{k}{2^{32}}\right)^3$ since we are checking three independent random values, which is astronomically small for any reasonable k!

Each evaluation of P or Q at a random point takes $\Theta(k)$ time, so the total verification time is $\Theta(k)$. So we have achieved a significant improvement over the naive[1] $\Theta(k^2)$ time.

Note that choosing 3 random values has no particular significance - we could choose any small constant number of random values to achieve this - it is a tradeoff between time and correctness probability.

6.3 Matrix Multiplication Verification

Suppose we are given matrices

$$A, B, C \in \mathbb{R}^{n \times n}$$

and wish to verify whether

$$AB = C$$

Computing the product explicitly requires matrix multiplication. Standard matrix multiplication takes

$$\Theta(n^3)$$

time, while the fastest known algorithms run in roughly

$$\Theta(n^{2.37})$$

time, though they are highly complicated and rarely used in practice.

6.3.1 Freivalds' Algorithm

Choose a random vector $X \in \{0,1\}^n$, aka of form

$$\begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}$$

where each x_i is an independent random bit.

Instead of comparing the matrices themselves, we compute ABX and CX. Since matrix-vector multiplication takes only $\Theta(n^2)$ and we are doing 3 such multiplications – $B \cdot X$, then $A \cdot (BX)$, and $C \cdot X$. Then we compare the two products entry by entry in $\Theta(n)$ time.

So the overall procedure is $\Theta(n^2)$ time.

The quantity MX may be viewed as a compact randomized hash of the matrix M. If two matrices differ, their hashes are unlikely to coincide for a random choice of X.

[1] There exists deterministic techniques to get this down to $\Theta(k \log k)$ time, but they have a large constant factor and also are very difficult to implement

More generally, one may choose k independent random vectors

$$X_1, X_2, \ldots, X_k$$

and define the hash of a matrix M as

$$(MX_1, MX_2, \ldots, MX_k)$$

Each additional random vector independently decreases the probability of error. Here k must be $\Theta(1)$ to preserve the overall $\Theta(n^2)$ time complexity.

6.3.2 Error Probability Analysis

$$\text{Let } M = AB - C$$

If $AB \neq C$, then $M \neq 0$. Freivalds' algorithm accepts incorrectly exactly when

$$MX = 0$$

Assume X is chosen uniformly from $\mathbb{F}^n$, where $\mathbb{F}$ is a finite set.
Since $M \neq 0$, we have

$$\dim(\ker M) \leq n - 1$$

The vector space $\mathbb{F}^n$ contains $|\mathbb{F}|^n$ vectors, while the nullspace of M contains at most $|\mathbb{F}|^{n-1}$ vectors.

$$\implies \Pr[MX = 0] \leq \frac{|\mathbb{F}|^{n-1}}{|\mathbb{F}|^n} = \frac{1}{|\mathbb{F}|}$$

Hence the probability that the algorithm gives false positive is at most

$$\frac{1}{|\mathbb{F}|}$$

So for the traditional choice of $\mathbb{F} = \{0, 1\}$, the error probability is at most $\frac{1}{2}$.
In practice, we can simply use 32-bit integers, which would give an error probability of at most $\frac{1}{2^{32}}$, which is negligible for all practical purposes. Of course, to be more sure we could run it multiple times independently...

6.4 A Matrix Problem

Problem Statement[2]**:** Given two $n \times m$ matrices A and B, determine if they are equivalent under row and column permutations. That is, can we permute the rows and columns of A to obtain B?
Additional Constraints: A and B both satisfy – the set of all entries is {1,2, ..., nm}.

Reduction: First we reduce the problem to a simpler one. If we consider R(M) to be the set of all R_i where each R_i is the set of values in the i-th row of a matrix M, and similarly C(M) then the matrices A and B are equivalent under row and column permutations if and only if R(A) = R(B) and C(A) = C(B).

[2]This question was taken from Codeforces 1980E

Proof of Reduction

Clearly, if A and B are equivalent under row and column permutations, then R(A) = R(B) and C(A) = C(B) since the row and column sets are preserved under these permutations. This proves that the set of all B such that R(B) = R(A) and C(B) = C(A) contains all matrices equivalent to A under row and column permutations (i.e. the set of all B such that R(B) = R(A) and C(B) = C(A) is a superset of the set of all B equivalent to A under row and column permutations).

Now notice that the number of row and column permutations on A are $n! \cdot m!$ (including the identity permutation). Each of these permutations gives a different matrix since the entries are all distinct. Also it can be shown that the number of matrices with given R(M) and C(M) is also $n! \cdot m!$ (In fact while proving this you will notice that the fact we are proving is quite intuitive)

Since the two sets have the same size and one contains the other, they must be equal. Hence R(A) = R(B) and C(A) = C(B) implies that A and B are equivalent under row and column permutations.

(Note that we assumed all entries are distinct in the above proof)

This reduced problem can be solved by two approaches:

6.4.1 Method 1: Set of Sets Approach (Row and Column sets)

Formally, let

$$R_i = \{a_{i1}, a_{i2}, \ldots, a_{im}\}$$

be the representation of row i. We compare:

$$\{R_1, R_2, \ldots, R_n\}_A = \{R_1, R_2, \ldots, R_n\}_B$$

and similarly for columns.

This method is conceptually exact but computationally heavy due to the need to compare complex set-of-sets structures. Set structures can be implemented using balanced binary trees in $\Theta(n \log n)$ time - In `C++` we have the `set` data structure which is implemented as a balanced binary tree. Comparing two sets of size m takes $\Theta(m \log m)$ time, and we have to do this for n rows and n columns, leading to a total time complexity of $\Theta(nm \log n \log m)$.

`C++` implementation:

```
//Matrices are 0-indexed and stored in 2D vectors "first" and "second"
set<set<int>>F;//set of sets for first matrix
for(int i=0;i<n;i++)
{
    set<int>temp(first[i].begin(),first[i].end()); //set of ith row
    F.insert(temp);
}

for(int i=0;i<n;i++)
{
    set<int>temp(second[i].begin(),second[i].end()); //set of ith row
    if (!F.count(temp)) //early exit if this row doesn't exist in the first matrix
    {
        cout<<"NO\n";
        return;
    }
}
```

6.4.2 Method 2: Hashing Based Verification of Rows and Columns

To obtain a more efficient solution, each row is hashed to a pair of values (sum, squaresum), where sum and squaresum denote the sum and sum of squares of elements in the row respectively. This representation is much easier to compare across rows.

Assuming the hash is sufficiently collision-resistant, we compare the sets of row hashes for both matrices. This can be implemented efficiently using `unordered_set` in `C++`, which provides expected $O(1)$ insertion and lookup time.

For each row i, we compute:

$$h(R_i) = \left(\sum_{j=1}^{m} a_{ij}, \sum_{j=1}^{m} a_{ij}^2 \right)$$

We define a custom hash for a pair (x, y) as:

$$H(x, y) = x \cdot 2^{32} + y$$

where x and y are the computed row statistics. This choice of pair hash is popular for 32 bit integers, as it combines the two values into a single 64-bit integer guaranteeing unique hashes.

We insert the hashes of all rows of matrix A into an unordered set and then check if the hashes of rows of matrix B are present in this set. If any hash from B is not found in the set from A, we can conclude that the matrices are not equivalent under row permutations.

Similarly for columns, we compute the column hashes and compare them in the same manner.

This method has an expected time complexity of $\Theta(nm)$ due to the linear time required to compute the hashes and the expected constant time for hash set operations.

This method involved hashing so the correctness of the algorithm is itself a random variable – but given the constraint that the matrix entries are from {1,2,. . . ,nm}, it seems highly unlikely that this will fail. In fact, it seems that this may be a unique hash in this case, but i am yet to find a proof for general (n,m).

`C++` implementation:

```
auto pair_hash=[](int x,int y){
    return ((x)<<32)+y;
};

unordered_set<long long>s;
loop(i,n)
{
    long long v1=0,v2=0;
    loop(j,m)
    {
        int x=first[i][j];
        v1+=x;
        v2+=x*x;
    }
    s.insert(pair_hash(v1,v2));
}
loop(i,n)
{
    long long v1=0,v2=0;
    loop(j,m)
```

```
    {
        int x=second[i][j];
        v1+=x;
        v2+=x*x;
    }
    if(!s.count(pair_hash(v1,v2))) //early exit if this row doesn't exist in the first matrix
    {
        cout<<"NO\n";
        return;
    }
}
```

6.5 A Sliding Window Problem

Problem Statement: Two integer arrays x and y, each of size n, are given along with an integer k ($1 \leq k \leq n$). For every contiguous subarray (window) of size k, determine whether the corresponding windows in x and y contain the same multiset of elements at every position.

More precisely, for every index (1-based) i such that $1 \leq i \leq n - k + 1$, we compare the multisets:

$$\{x_i, x_{i+1}, \ldots, x_{i+k-1}\} \quad \text{and} \quad \{y_i, y_{i+1}, \ldots, y_{i+k-1}\}$$

and check whether they are identical for all i.

Rolling Hash Method

To obtain an efficient solution, we maintain two rolling hashes for each window:

$$S = \sum a_i, \quad Q = \sum a_i^2$$

These values can be updated in $\mathcal{O}(1)$ time when the window slides by removing the outgoing element and adding the incoming element.

For each position i, we compute:

$$(S_x(i), Q_x(i)) \quad \text{and} \quad (S_y(i), Q_y(i))$$

and check whether:

$$(S_x(i), Q_x(i)) = (S_y(i), Q_y(i))$$

If the condition holds for all i, we conclude that every corresponding window has identical multiset structure.

C++ Implementation

```
long long sumX=0,sumY=0;
long long sumX2=0,sumY2=0;

for(int i=0;i<k;i++)
{
    sumX+=x[i];
    sumY+=y[i];
    sumX2+=x[i]*x[i];
    sumY2+=y[i]*y[i];
}
```

```
bool ok=1;
for(int i=k;i<n;i++)
{
    if(!((sumX==sumY)&&(sumX2==sumY2)))
    {
        ok=0;
        break;
    }

    sumX-=x[i-k];
    sumX+=x[i];
    sumX2-=x[i-k]*x[i-k];
    sumX2+=x[i]*x[i];

    sumY-=y[i-k];
    sumY+=y[i];
    sumY2-=y[i-k]*y[i-k];
    sumY2+=y[i]*y[i];
}
cout<<(ok?"YES":"NO");
```

This method is very easy to implement and runs in $\Theta(n)$ time, which is efficient for this problem. What was especially nice about this method is that we can maintain the hash values in constant time as the window slides, which is what I meant by using a **rolling hash**. However, it relies on the assumption that the hash values are unique for different multisets. We can improve this (if needed) by doing a second round of hashing with a different hash function like polynomial hashing.

Chapter 7

The Probabilistic Method

7.1 The Dominating Set Problem

Problem Statement: Given a graph:

$$G = (V, E)$$

find a smallest possible subset:

$$D \subseteq V$$

such that every vertex in the graph is either:

- contained in D, or
- adjacent to some vertex in D.

Such a set is called a *dominating set*.

Intuitively, we may think of the vertices in D as "control centers" or "facilities" such that every location in the graph is either itself a facility or directly connected to one.

Unfortunately, finding the minimum dominating set is NP-hard, so we want to find a polynomial time algorithm that gives us a reasonably small[1] dominating set.

A Randomised Solution

Suppose the graph has minimum degree d. We now consider the following extremely simple randomised strategy:

> Choose k vertices randomly out of the n vertices (uniformly among the $\binom{n}{k}$ possible choices). If the chosen set forms a dominating set, output it. Otherwise, try again!

At first glance, this algorithm seems dumb. However, it turns out that if we choose k large enough, the probability of failure becomes small. The key question thus becomes:

> What is the probability of failure as a function of k, n, and d?

We now analyse this probability.

[1] We care more about the asymptotic behaviour of the size of set produced by the algorithm and less about which polynomial time complexity the algorithm has.

Probability That a Vertex Remains Undominated

Fix a vertex $v \in V$. Since the graph has minimum degree d, $N[v]$ contains at least $d+1$ vertices. For v to remain undominated, none of the selected k vertices may lie inside $N[v]$

When choosing vertices uniformly at random, the probability that a single chosen vertex does *not* belong to $N[v]$ is at most:

$$1 - \frac{d+1}{n}$$

$$\implies \Pr[v \text{ remains undominated}] \leq \left(1 - \frac{d+1}{n}\right)^k$$

Using the inequality[2] $1 - x \leq e^{-x}$ for all $x \in \mathbb{R}$, we obtain:

$$\Pr[v \text{ remains undominated}] \leq e^{-k(d+1)/n}$$

Bounding the Failure Probability

The algorithm fails if at least one vertex remains undominated.

Applying the union bound[3]

$$\Pr[\text{failure}] \leq n \cdot e^{-k(d+1)/n}$$

We would like this probability to be strictly less than 1. Therefore we require:

$$ne^{-k(d+1)/n} < 1$$

$$\implies \ln n - \frac{k(d+1)}{n} < 0$$

$$\implies k > \frac{n \ln n}{d+1}$$

Thus, choosing:

$$k = \frac{cn \ln n}{d+1}$$

for any constant $c > 1$ gives $\Pr[\text{failure}] \leq n^{1-c}$ which tends to 0 as $n \to \infty$.

What Does This Actually Mean?

The important point is not merely that the algorithm succeeds with high probability.

Rather, we have shown something stronger –

> There exists a dominating set[4] of size roughly:
>
> $$\frac{n \log n}{d}$$
>
> Indeed, if no such dominating set existed, our algorithm could never possibly succeed.
>
> Thus a simple probabilistic argument simultaneously:

- proves the existence of a reasonably small dominating set,
- and gives a randomised algorithm capable of finding one.

[2] Refer Appendix B.1

[3] Refer Appendix A.2

[4] This method has a limitation - if $d < c \ln n$, then the size of the dominating set produced by the algorithm is basically the whole set of vertices, making it useless.

Amplifying the Success Probability

Even if a single run fails, we may simply repeat the algorithm independently.

If the failure probability of one run is $\boldsymbol{p} = \frac{1}{n^{c-1}}$, then after $\boldsymbol{t}$ independent repetitions, the probability that *every* run fails equals $\boldsymbol{p^t}$ which decreases exponentially fast.

So the tradeoff is choosing a larger c has a smaller expected number of repetitions $(\frac{n^{c-1}}{n^{c-1}-1})^5$ but a larger dominating set, while choosing a smaller c has a larger expected number of repetitions but a smaller dominating set.

7.2 Bipartite Subgraph

Problem statement. Given an undirected graph $G = (V, E)$ with $|E| = m$, we wish to find a bipartition of V that maximizes the number of edges between vertices of different partitions. Equivalently, we want a 2-coloring of the vertices that maximizes the number of bichromatic edges. We aim to show that we can always guarantee at least $m/2$ edges in the cut.

Expected Number of Bichromatic Edges

Let X be the number of bichromatic edges under a random 2-coloring of the vertices. Each edge has a probability $\frac{1}{2}$ of being bichromatic, so define the indicator variable I_i as 1 when the edge numbered i is bichromatic and 0 otherwise. We can conclude that for a random vertex 2-coloring, $\Pr(I_i = 1)=\frac{1}{2}$ and $\Pr(I_i = 0)=\frac{1}{2}$, so $\mathbb{E}[I_i] = \frac{1}{2}$. Since $X = \sum_{i=1}^{m} I_i$, we have:

$$\mathbb{E}[X] = \mathbb{E}[\sum_{i=1}^{m} I_i] = \sum_{i=1}^{m} \mathbb{E}[I_i] = \frac{m}{2}$$

A Weak Lower Bound

Let $X \in \{0, 1, \ldots, m\}$ be the number of bichromatic edges under a random 2-coloring of the vertices. We already have $\mathbb{E}[X] = \frac{m}{2}$.

We derive a crude lower bound on $\Pr\big(X \geq \frac{m}{2}\big)$.

Case 1: $m = 2k$ **(even).**

Let $p = \Pr(X \geq k)$. The maximum value of $E(X)$ under these constraints would be achieved when we push all mass in $\{0, 1, \ldots, k-1\}$ to $k-1$, and all mass in $\{k, \ldots, 2k\}$ to $2k$. Then

$$\mathbb{E}[X] \leq (1-p)(k-1) + p(2k).$$

Using $\mathbb{E}[X] = k$,

$$\begin{aligned} k &\leq (1-p)(k-1) + 2kp \\ k &\leq k - 1 + p(k+1) \\ 1 &\leq p(k+1) \\ p &\geq \frac{1}{k+1} = \frac{2}{m+2}. \end{aligned}$$

$$\implies \Pr\Big(X \geq \frac{m}{2}\Big) \geq \frac{2}{m+2}.$$

[5] Since the probability of failure is $p = (n^{c-1} - 1)/n^{c-1}$ and this is a bernoulli trial, the expected number of trials for success is 1/p

Case 2: $m = 2k + 1$ **(odd).**

Let $p = \Pr(X \geq k+1)$. The maximum value of $E(X)$ under these constraints would be achieved when all mass in $\{0, 1, \ldots, k\}$ is pushed to k, and all mass in $\{k+1, \ldots, 2k+1\}$ is pushed to $2k + 1$. Then

$$\mathbb{E}[X] \leq (1-p)k + p(2k+1).$$

Using $\mathbb{E}[X] = k + \frac{1}{2}$,

$$k + \frac{1}{2} \leq k + p(k+1)$$

$$\frac{1}{2} \leq p(k+1)$$

$$p \geq \frac{1}{2(k+1)} = \frac{1}{m+1}.$$

$$\implies \Pr\Big(X \geq \frac{m}{2}\Big) \geq \frac{1}{m+1}.$$

Conclusion –

$$\Pr\Big(X \geq \frac{m}{2}\Big) \geq min(\frac{2}{m+2}, \frac{1}{m+1}) = \frac{1}{m+1}.$$

A Simple Randomised Algorithm

From the probabilistic analysis, a random coloring satisfies

$$\Pr\left(X \geq \frac{m}{2}\right) \geq \frac{1}{m+1}.$$

Therefore, if we repeatedly sample random colorings independently, the probability that none of T trials succeeds is at most

$$(1 - \frac{1}{m+1})^T.$$

Choosing $T = (m+1)\ln(m+1)$ gives

$$(1 - \frac{1}{m+1})^T \leq e^{-\frac{T}{m+1}} = e^{-\ln(m+1)} = \frac{1}{m+1}$$

Which goes to 0 as $m \to \infty$.

While simple, this method does not exploit structure of intermediate solutions and relies purely on repeated random sampling rather than improvement steps.

Chapter 8

Heuristic Algorithms

Many computational problems of practical interest have a statement like "Given a graph find a set of vertices of size atmost x satisfying property P."
Now one way to guarantee a solution is to find the minimum sized set of vertices satisfying P, but this is usually not doable in polynomial time.
So we are often forced to settle for a solution that is not necessarily optimal but can be found efficiently. Such algorithms are *heuristic algorithms* – algorithms designed to quickly find solutions that are often good in practice, even if they do not always provide provable optimality guarantees or even any guarantees at all.
Inherently the analysis of heuristic algorithms involve probability and expectation, making them a natural application of the tools we have developed so far.

8.1 The Dominating Set Problem Revisited

Recall the dominating set problem: given a graph $G = (V, E)$, find a smallest possible subset $D \subseteq V$ such that every vertex in the graph is either contained in D or adjacent to some vertex in D.

A Greedy Heuristic

A very natural approach is the following greedy algorithm:

> Repeatedly select the vertex that dominates the largest number of currently undominated vertices.

More precisely:

1. Initially, all vertices are marked undominated.
2. At each step, choose a vertex whose closed neighbourhood $N[v]$ contains the largest number of undominated vertices.
3. Add this vertex to the dominating set.
4. Mark all vertices in $N[v]$ as dominated.
5. Repeat until every vertex becomes dominated.

At every step we greedily maximise immediate progress.

Analysing the Greedy Heuristic

Let OPT denote the size of a minimum dominating set.

Suppose that at some stage of the algorithm there remain r undominated vertices.

Since the optimal solution uses only OPT vertices to dominate the entire graph, those same OPT vertices must also collectively dominate these remaining r vertices.

Therefore, by the pigeonhole principle, at least one of those optimal vertices must dominate at least $\frac{r}{\mathrm{OPT}}$ of the currently undominated vertices.

But the greedy algorithm always chooses a vertex dominating as many undominated vertices as possible. Hence the greedy step must dominate at least $\frac{r}{\mathrm{OPT}}$ new vertices.

Thus after one greedy step, the number of remaining undominated vertices becomes at most

$$r - \frac{r}{\mathrm{OPT}} = r\left(1 - \frac{1}{\mathrm{OPT}}\right)$$

If r_t denotes the number of undominated vertices after t greedy steps, then

$$r_{t+1} \leq r_t \left(1 - \frac{1}{\mathrm{OPT}}\right)$$

Since initially $r_0 = n$, we obtain

$$r_t \leq n\left(1 - \frac{1}{\mathrm{OPT}}\right)^t$$

We would like all vertices to become dominated, i.e. $r_t < 1$.

Since

$$r_t \leq n\left(1 - \frac{1}{\mathrm{OPT}}\right)^t$$

it suffices to find the smallest t such that

$$n\left(1 - \frac{1}{\mathrm{OPT}}\right)^t < 1$$

Rearranging:

$$\left(1 - \frac{1}{\mathrm{OPT}}\right)^t < \frac{1}{n}$$

Taking logarithms:

$$t \ln\left(1 - \frac{1}{\mathrm{OPT}}\right) < -\ln n$$

Since $\ln(1 - \frac{1}{\mathrm{OPT}}) < 0$, dividing reverses the inequality:

$$t > \frac{\ln n}{-\ln\left(1 - \frac{1}{\mathrm{OPT}}\right)}$$

Let S denote the size of the dominating set produced by the greedy algorithm. Then S is the smallest integer t satisfying the above inequality. Thus:

$$S = \left\lceil \frac{\ln n}{-\ln\left(1 - \frac{1}{\mathrm{OPT}}\right)} \right\rceil \leq \frac{\ln n}{-\ln\left(1 - \frac{1}{\mathrm{OPT}}\right)} + 1$$

Now using the inequality

$$-\ln(1 - x) \geq x$$

with $x = \frac{1}{\text{OPT}}$, we obtain

$$-\ln\left(1 - \frac{1}{\text{OPT}}\right) \geq \frac{1}{\text{OPT}}$$

Taking reciprocals reverses the inequality:

$$\frac{1}{-\ln\left(1 - \frac{1}{\text{OPT}}\right)} \leq \text{OPT}$$

Therefore:

$$S \leq \ln n \cdot \frac{1}{-\ln\left(1 - \frac{1}{\text{OPT}}\right)} + 1$$

$$\implies S \leq \text{OPT} \ln n + 1$$

Hence the greedy algorithm produces a dominating set of size at most

$$\boxed{\text{OPT} \ln n + 1}$$

8.2 Exam Scheduling

Problem Statement: We are given a set of exams and a set of students. Each student is enrolled in a subset of the exams. We wish to schedule as many exams as possible in the first time slot[1], subject to the constraint that no student should have two exams scheduled at the same time[2].

The goal is to find a largest possible subset of exams that can be scheduled simultaneously without any conflict.

Reduction

We model the problem using a graph $G = (V, E)$ where:

- each vertex represents an exam,
- an edge exists between two vertices if there is at least one student enrolled in both corresponding exams.

In this formulation, a feasible schedule corresponds to a set of vertices with no edges between them. Such a set is called an *independent set*.

Thus, the problem reduces to finding a maximum independent set in a graph.

8.2.1 Random Permutation

V is the set of all vertices. Maintain a set - the independent set we are building (S). Pick a random vertex from V – delete it from V and if none of its neighbours are in S, put it in S. Keep doing this until V is empty. This guarantees forming a valid independent set.

Note that the order in which we pick vertices is a uniformly random permutation of the vertices.

[1] this is different from the problem of the overall scheduling problem which is equivalent to finding the Chromatic Number. This is a simpler problem where we only care about the first time slot.

[2] Or you could just give them a Time Turner and hope for the best :)

Probability a Vertex is Selected

Fix a vertex v. Consider the set consisting of v and its neighbours $N(v)$. There are $d(v)+1$ vertices in total.

Since all relative orderings are equally likely, the probability that v is the picked earliest among these vertices is:

$$\Pr[v \in S] = \frac{1}{d(v)+1}$$

Expected Size of the Independent Set

Let I_v be the indicator random variable for the event $v \in S$. Then:

$$|S| = \sum_{v \in V} I_v$$

Taking expectation and using linearity:

$$\mathbb{E}[|S|] = \sum_{v \in V} \mathbb{E}[I_v] = \sum_{v \in V} \frac{1}{d(v)+1}$$

Therefore, there exists an independent set of size at least:

$$\sum_{v \in V} \frac{1}{d(v)+1}$$

Denoting the maximum independent set size by $\alpha(G)$, we have:

$$\alpha(G) \geq \sum_{v \in V} \frac{1}{d(v)+1}$$

Using the Jensen's[3] inequality on 1/(x+1), we have:

$$\sum_{v \in V} \frac{1}{d(v)+1} \geq \frac{n}{d+1}$$

where d is the average degree of the graph. Hence:

$$\alpha(G) \geq \frac{n}{d+1}$$

8.2.2 Local Decision

We now describe an alternative randomized construction that produces a large independent set via local random choices.

Each vertex independently chooses a random value:

$$X_v = \begin{cases} 1 & \text{with probability } p \\ 0 & \text{with probability } 1-p \end{cases}$$

We construct a set S by including a vertex v if:

- $X_v = 1$, and
- for every neighbour $u \in N(v)$, $X_u = 0$.

[3] Refer Appendix B.2

Probability of Inclusion

Fix a vertex v. The probability that v is included in S is:

$$\Pr[v \in S] = p(1-p)^{d(v)}$$

Therefore:

$$\mathbb{E}[|S|] = \sum_{v \in V} p(1-p)^{d(v)}$$

Guarantee on Expected Size

We analyze the randomized construction where each vertex is selected independently with probability p, and a vertex v is included in the independent set S if it is selected and none of its neighbors are selected. Then:

$$\mathbb{E}[|S|] = \sum_{v \in V} p(1-p)^{d(v)}.$$

Using convexity of $f(x) = (1-p)^x$ and Jensen's inequality, we obtain:

$$\sum_{v \in V} (1-p)^{d(v)} \geq n(1-p)^d,$$

where $d = \frac{1}{n}\sum_{v \in V} d(v)$ is the average degree. Hence:

$$\mathbb{E}[|S|] \geq np(1-p)^d.$$

We now optimize the function:

$$f(p) = p(1-p)^d.$$

Differentiating:

$$f'(p) = (1-p)^d - pd(1-p)^{d-1} = (1-p)^{d-1}\big((1-p) - pd\big).$$

Setting $f'(p) = 0$, we obtain:

$$1 - p - pd = 0 \quad \Rightarrow \quad p = \frac{1}{d+1}.$$

Substituting this optimal value gives:

$$\mathbb{E}[|S|] \geq n \cdot \frac{1}{d+1}\left(1 - \frac{1}{d+1}\right)^d = \frac{n}{d+1}\left(\frac{d}{d+1}\right)^d.$$

Using the standard bound $\left(1 - \frac{1}{d+1}\right)^d \geq e^{-d/(d+1)}$, we obtain:

$$\mathbb{E}[|S|] \geq \frac{n}{d+1} \cdot e^{-d/(d+1)}. \geq \frac{n}{e(d+1)}.$$

This is a slightly weaker bound than the previous method, but it has the advantage of being local and easily parallelizable, as each vertex makes its decision independently based on local information. This is used in scheduling processes in a distributed system, where each process (vertex) decides whether to execute based on its own state and the states of its neighbors.

8.3 Bipartite Subgraph Revisited

Recall the problem of finding a bipartition of the vertices that maximizes the number of bichromatic edges. We showed that a random coloring achieves at least $m/2$ bichromatic edges in expectation, and with some probability. Now we refine our approach by showing that we can always find a coloring with at least $m/2$ bichromatic edges via a self-correction procedure.

Self-Correction

Start with a random coloring of the vertices. If it has at least $m/2$ bichromatic edges, we are done.

Otherwise, let X be the number of bichromatic edges. We have $X < m/2$, so there are more monochromatic edges than bichromatic edges. Call a vertex v *bad* if it has more monochromatic incident edges than bichromatic incident edges. Otherwise it is *good*.

Claim: If the current cut has fewer than $m/2$ crossing edges, then a bad vertex must exist.

Proof idea: Each monochromatic edge contributes equally to two endpoints. So total number of monochromatic edges is half the sum of monochromatic degrees of vertices. If globally there are more monochromatic edges than crossing ones, by pigeonhole principle[4], some vertex must be incident to more monochromatic than bichromatic edges.

Now consider flipping the color of a bad vertex v. Let $d_h(v)$ be the number of monochromatic (same-color) edges incident to v and $d_c(v)$ the number of bichromatic edges. Since $d_h(v) > d_c(v)$, flipping v increases X by

$$\Delta X = d_h(v) - d_c(v) > 0.$$

Thus, each flip strictly increases the number of bichromatic edges.

Pseudocode:

```
initialize random coloring of vertices
compute X

while exists bad vertex v:
    flip color of v
    update X locally
```

Complexity analysis: Each flip increases X by at least 1, and $X \leq m$, so there are at most m flips.

If we maintain for each vertex the counts of monochromatic and bichromatic incident edges, each flip can be updated in $O(\deg(v))$. To find a bad vertex we need atmost $O(n)$ time. Over all flips, total time is atmost

$$O\left(\sum_{v \text{ flipped}} (\deg(v) + n)\right) = O(mn + m).$$

Hence the full procedure runs in atmost $O(mn + m)$[5] after initialization.

[4] You may also prove this by contradiction – if all vertices are bad, then the total number of monochromatic edges is more than the total number of bichromatic edges by summing the inequality, contradicting the assumption.

[5] this can be improved to $O(m \log n)$ by using priority queues with lazy updates – its a fun competitive programming exercise, I encourage you to try it!

Appendix A

Probability and Expectation

This appendix collects a small set of probabilistic results that are used throughout the book. The goal is not to develop probability theory, but to record a minimal toolkit for algorithmic analysis.

A.1 Linearity of Expectation

For any random variables $X_1, X_2, \ldots, X_n$ (not necessarily independent),

$$\mathbb{E}\left[\sum_{i=1}^{n} X_i\right] = \sum_{i=1}^{n} \mathbb{E}[X_i].$$

This holds regardless of dependence between variables.
In particular, for indicator variables $I_1, \ldots, I_n$,

$$\mathbb{E}\left[\sum_{i=1}^{n} I_i\right] = \sum_{i=1}^{n} \Pr(I_i = 1).$$

A.2 Union Bound

For any events $A_1, A_2, \ldots, A_n$,

$$\Pr\left(\bigcup_{i=1}^{n} A_i\right) \leq \sum_{i=1}^{n} \Pr(A_i).$$

This is often used to upper bound the probability that at least one "bad event" occurs.

A.3 Variance

For a random variable X,

$$\mathrm{Var}(X) = \mathbb{E}[X^2] - (\mathbb{E}[X])^2 .$$

For any two random variables X and Y,

$$\mathrm{Var}(X + Y) = \mathrm{Var}(X) + \mathrm{Var}(Y) + 2\,\mathrm{Cov}(X, Y),$$

where

$$\mathrm{Cov}(X, Y) = \mathbb{E}[XY] - \mathbb{E}[X]\mathbb{E}[Y].$$

If X and Y are independent, then $\mathrm{Cov}(X, Y) = 0$, and hence

$$\mathrm{Var}(X + Y) = \mathrm{Var}(X) + \mathrm{Var}(Y).$$

More generally, for any collection $X_1, \ldots, X_n$,

$$\mathrm{Var}\left(\sum_{i=1}^{n} X_i\right) = \sum_{i=1}^{n} \mathrm{Var}(X_i) + 2 \sum_{1 \le i < j \le n} \mathrm{Cov}(X_i, X_j).$$

A.4 Indicator Variables

For an event A, define the indicator random variable

$$I_A = \begin{cases} 1 & \text{if } A \text{ occurs,} \\ 0 & \text{otherwise.} \end{cases}$$

Then:

$$\mathbb{E}[I_A] = \Pr(A).$$

Indicator variables are commonly used to convert counting problems into sums of random variables.

Appendix B

Inequalities

B.1 Exponential Inequality

For any $x \in \mathbb{R}$,

$$1 - x \leq e^{-x}.$$

Consequently, for $x \in [0, 1]$ and integer $k \geq 0$,

$$(1 - x)^k \leq e^{-kx}.$$

This is frequently used to simplify probability decay expressions.

B.2 Jensen's Inequality

For a convex function f and random variable X,

$$f(\mathbb{E}[X]) \leq \mathbb{E}[f(X)].$$

This is often used to derive bounds on expectations of functions of random variables.

Another form of Jensen's inequality is for a convex function f and real numbers $x_1, x_2, \ldots, x_n$,

$$f(\frac{1}{n}\sum_{i=1}^{n} x_i) \leq \frac{1}{n}\sum_{i=1}^{n} f(x_i).$$

For concave functions, the inequality is reversed.

Appendix C

Discrete Mathematics

C.1 Pigeonhole Principle

Let $a_1, a_2, \ldots, a_n$ be non-negative real numbers and let

$$S = \sum_{i=1}^{n} a_i.$$

Then there exists some index i such that

$$a_i \geq \frac{S}{n}.$$

Interpretation

This simply states that at least one value is at least the average value.

Proof by Contradiction form:

If every $a_i < \frac{S}{n}$, then summing over all i gives

$$\sum_{i=1}^{n} a_i < S,$$

which is a contradiction.

C.2 Basic Graph Definitions

A graph is an ordered pair

$$G = (V, E)$$

where V is a set of vertices and $E \subseteq \{\{u, v\} \mid u, v \in V,\ u \neq v\}$ is a set of edges.

Degree

For a vertex $v \in V$, the degree $d(v)$ is the number of edges incident to v.

The average degree of a graph is:

$$d = \frac{1}{n} \sum_{v \in V} d(v)$$

where $n = |V|$.

Bipartite Graph

A graph $G = (V, E)$ is bipartite if the vertex set V can be partitioned into two disjoint sets L and R such that:

$$V = L \cup R, \quad L \cap R = \emptyset$$

and every edge connects a vertex in L to a vertex in R. That is, there are no edges with both endpoints in the same set.

Appendix D

Linear Algebra

D.1 Kernel and Rank

For a matrix

$$M \in \mathbb{F}^{m \times n}$$

the kernel (or nullspace) of M is defined as

$$\ker(M) = \{x \in \mathbb{F}^n : Mx = 0\}$$

That is, the kernel consists of all vectors mapped to the zero vector by M.

The rank of M, denoted

$$\operatorname{rank}(M)$$

is the dimension of the column space of M, equivalently the maximum number of linearly independent columns of M.

The dimension of the kernel is called the nullity of M, written as

$$\operatorname{nullity}(M) = \dim(\ker(M))$$

These quantities satisfy the rank-nullity theorem:

$$\operatorname{rank}(M) + \operatorname{nullity}(M) = n$$

for every matrix

$$M \in \mathbb{F}^{m \times n}$$

D.2 Special Cases

For the zero matrix

$$0 \in \mathbb{F}^{m \times n}$$

every vector satisfies

$$0x = 0$$

Hence,

$$\ker(0) = \mathbb{F}^n$$

and therefore

$$\operatorname{nullity}(0) = n, \qquad \operatorname{rank}(0) = 0$$

On the other hand, if

$$M \neq 0$$

then at least one column of M is nonzero, implying

$$\text{rank}(M) \geq 1$$

Applying the rank-nullity theorem gives

$$\text{nullity}(M) \leq n - 1$$

Equivalently,

$$\dim(\ker(M)) \leq n - 1$$

Thus the kernel of a nonzero matrix forms a proper subspace of $\mathbb{F}^n$.